

VIETNAM NATIONAL UNIVERSITY-HO CHI MINH CITY

HO CHI MINH UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

KNOWLEDGE ENGINEERING DEPARTMENT

---

## Homework 5

---

Course: Introduction to Artificial Intelligence

*Student:*

23127165 - Nguyen Hai Dang

*Instructors:*

Mr Bui Tien Len

July 26, 2025



## Contents

|          |                                                                            |          |
|----------|----------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Problem 1: Map Coloring with Backtracking Search</b>                    | <b>2</b> |
| 1.1      | Draw the Constraint Graph . . . . .                                        | 2        |
| 1.2      | Solution by Using Backtracking Search with Appropriate Heuristics. . . . . | 2        |
| <b>2</b> | <b>Problem 2: Constraint Propagation Techniques</b>                        | <b>4</b> |
| 2.1      | Cross out all values will be eliminated with Foward Checking . . . . .     | 4        |
| 2.2      | Cross out all values will be eliminated Arc Consistency . . . . .          | 4        |

# 1 Problem 1: Map Coloring with Backtracking Search

## 1.1 Draw the Constraint Graph

- **Variables:**  $X = \{A, B, C, D, E, F, G\}$
- **Domains:**  $D_i = \{\text{Red (R)}, \text{Green (G)}, \text{Blue (B)}\} \quad \forall i \in X$
- **Constraints:** Neighboring variables must not have the same color. These are binary constraints:  
 $\{(A, B), (A, C), (A, D), (A, E), (B, C), (B, D), (C, D), (D, E), (D, F), (D, G), (E, F), (E, G), (F, G)\}$

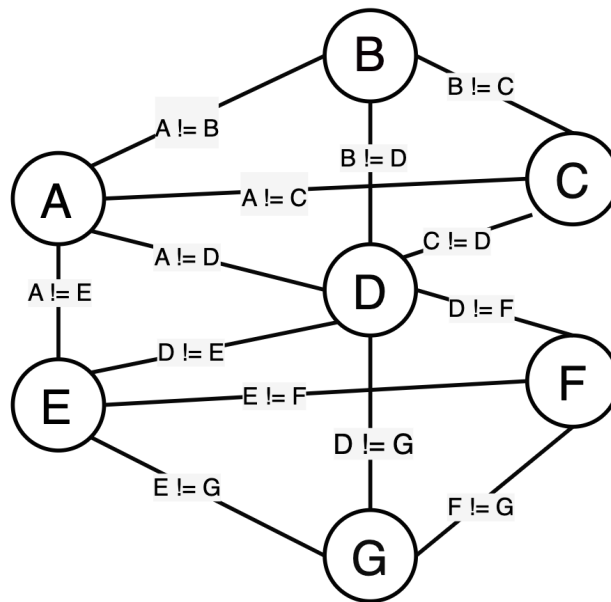


Figure 1: The constraint graph of the corresponding map

## 1.2 Solution by Using Backtracking Search with Appropriate Heuristics.

**Search Strategy:** We use backtracking with intelligent variable ordering heuristics:

- **Most Constraining Variable (MCV):** Select variables with the fewest remaining legal values (also known as MRV — Minimum Remaining Values).
- **Degree Heuristic:** Break ties by choosing variables with the highest degree (i.e., most constraints on remaining unassigned variables).
- **Least Constraining Value (LCV):** When selecting a value (color) for a variable, choose the one that rules out the fewest values in the domains of neighboring variables.

**Search Process:**

1. Initially, all variables have the same number of remaining colors. We select node **D** using the **Degree Heuristic**, as it is the most connected node.
2. After assigning a color to **D**, nodes **A** and **E** have the same number of remaining values, the same degree, and impose the same level of constraint on others. We choose **A** based on **alphabetical order** as a tie-breaker.
3. At this point, nodes **B**, **C**, and **E** have the same number of remaining values. We select **E**, again using the **Degree Heuristic**.
4. Now, we consider the remaining unassigned nodes: **B**, **C**, **F**, and **G**. In this configuration:
  - **B** and **C** are adjacent and each has only one remaining color.
  - **F** and **G** are also adjacent and each has only one remaining color.

This leads to a conflict where no valid color assignment is possible for both pairs. Therefore, the problem **has no valid solution** under this assignment path.

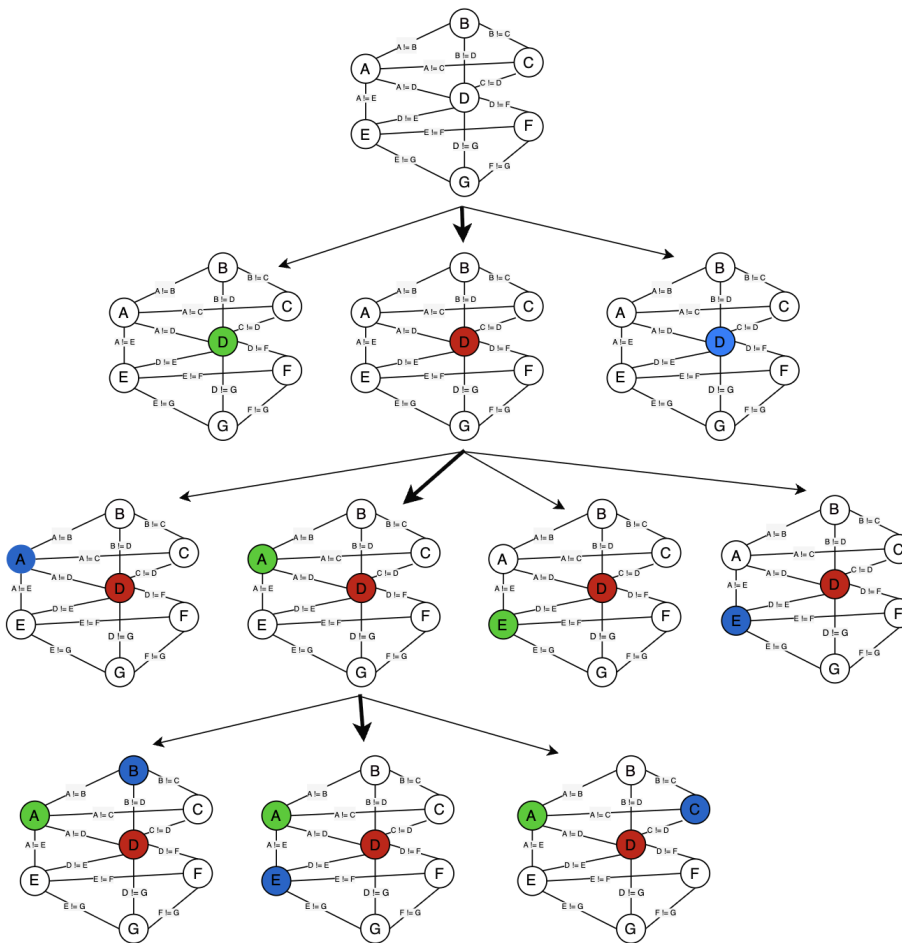


Figure 2: Search tree showing coloring decisions and conflicts

**Theoretical Analysis - Graph Coloring Theorem:** Furthermore, the impossibility of finding a solution can be proven using graph theory principles:

- Theorem: If a graph  $G$  contains a complete subgraph  $K_n$  (clique of size  $n$ ), then  $G$  requires at least  $n$  colors.
- Application: In our constraint graph, regions A, B, C, D form a complete subgraph  $K_4$ , meaning all four regions are mutually adjacent. Since we only have 3 colors available Red, Green, Blue, but need at least 4 colors for a  $K_4$  subgraph, this map is mathematically impossible to color.

**Conclusion:** The CSP has no solution due to the chromatic number requirement exceeding available colors.

## 2 Problem 2: Constraint Propagation Techniques

### 2.1 Cross out all values will be eliminated with Forward Checking

**Scenario:** Variable NT is assigned the value **Green (G)**.

**Forward Checking Process** Assigning  $NT = G$  triggers constraint propagation. We eliminate the value  $G$  from the domains of all neighboring variables to maintain consistency (no two adjacent regions can share the same color).

**NT's Neighbors:** { AL, BC, NU, SA, YU }

| AL                    | BC                    | MA        | NT           | NU                    | SA                    | YU                    |
|-----------------------|-----------------------|-----------|--------------|-----------------------|-----------------------|-----------------------|
| {R, <del>G</del> , B} | {R, <del>G</del> , B} | {R, G, B} | { <b>G</b> } | {R, <del>G</del> , B} | {R, <del>G</del> , B} | {R, <del>G</del> , B} |

**Result:** Green is successfully eliminated from all of NT's neighbors. Domain reduction helps avoid future conflicts and may lead to early failure detection.

### 2.2 Cross out all values will be eliminated Arc Consistency

**Initial State:** Assigned variables:  $AL = B$ ,  $MA = R$ . Others remain unassigned.

**AC-3 Algorithm Execution:** Arc consistency ensures that for every variable, each value in its domain has at least one consistent value in each neighboring variable's domain.

| AL           | BC                                 | MA           | NT                                | NU                                 | SA                                 | YU                                 |
|--------------|------------------------------------|--------------|-----------------------------------|------------------------------------|------------------------------------|------------------------------------|
| { <b>B</b> } | { <del>R</del> , G, <del>B</del> } | { <b>R</b> } | {R, <del>G</del> , <del>B</del> } | { <del>R</del> , <del>G</del> , B} | { <del>R</del> , G, <del>B</del> } | { <del>R</del> , <del>G</del> , B} |